

Hi3519DV500 Linux 内存扩展至 256MB — Display 子系统刷屏问题修复指南

一、问题现象

Hi3519DV500 开发板物理内存 2GB，默认 bootargs 中 `mem=132928K`（约 130MB）分配给 Linux。当将 Linux 可用内存扩展到 `mem=256M` 后，串口终端被以下错误疯狂刷屏：

```
01-01 00:01:46.610 967 967 E 02500/HDF_DISP: layer is not opened
01-01 00:01:46.610 967 967 E 02500/HDF_DISP: layer is not opened
01-01 00:01:46.621 967 967 E 02500/HDF_DISP: layer is not opened
[hilog ringbuffer full, drop 48787 line(s) log]
```

系统基本可用（内存分配正常），但串口输出被淹没，无法正常使用。

二、根因分析

2.1 错误来源

错误信息 `layer is not opened` 来自 SoC display HAL 层的 `CHECK_LAYER_OPENED` 宏：

文件：`device/soc/hisilicon/common/hal/display/include/display_layer_internal.h`

```
#define CHECK_LAYER_OPENED(fd, ret) do { \
    if ((fd) == -1) { \
        HDF_LOGE("layer is not opened"); \
        return (ret); \
    } \
} while (0)
```

该宏在 `display_layer.c` 的 6 个函数中被调用：`CloseLayer` (825行)、`SetLayerVisible` (848行)、`SetLayerDirtyRegion` (872行)、`Flush` (895行)、`GetLayerBuffer` (931行)、`waitForVBlank` (965行)。

2.2 完整调用链

```
window_manager_lite (wms.cpp:35)
→ HiFbdevInit()
→ DisplayInit() → LayerInitialize()           // 注册 SoC display 函数指针
→ OpenLayer() → CreateGraphicLayer()          // 打开 /dev/fb0 失败，fd 保持 -1
→ SetLayerVisible(true)                       // → CHECK_LAYER_OPENED → 报错
→ AllocDisplayBuffer()                        // → CHECK_LAYER_OPENED → 报错

渲染循环 (lite_wm.cpp:515)
→ LcdFlush() → Flush()                        // → CHECK_LAYER_OPENED → 持续刷屏
```

关键代码文件：

文件	作用
foundation/window/window_manager_lite/services/wms/wms.cpp:35	调用 <code>HiFbdevInit()</code> 启动 display
foundation/graphic/graphic_utils_lite/frameworks/hals/hi_fbdev.cpp:97	调用 <code>LayerInitialize()</code> 获取函数指针
device/soc/hisilicon/common/hal/display/display_layer.c	SoC display HAL 实现 (MPI 硬件操作)
foundation/graphic/graphic_utils_lite/BUILD.gn:154	通过 <code>-ldisplay_layer</code> 链接 SoC HAL

2.3 为什么 mem=132928K 不刷屏， mem=256M 刷屏

- mem=132928K (130MB) 时 Linux 内存极度紧张， window_manager_lite 无法完整启动渲染循环，所以不会触发持续刷屏
- mem=256M 时内存充裕， window_manager_lite 成功启动并进入渲染循环，但 CreateGraphicLayer() 因 display 硬件未就绪而失败 (fd=-1)，渲染循环每次迭代都触发 CHECK_LAYER_OPENED 报错

2.4 第一次修复失败的原因 (preload=2 无效)

第一次尝试将 device_info.hcs 中 display 设备的 preload 设为 2 (禁用加载)，但 无效。原因：
display_layer HAL 不是通过 HDF 设备框架加载的。它是 graphic_utils_lite 在 BUILD.gn 中通过 -ldisplay_layer 链接器标志直接链接的共享库。HCS 中的 preload 设置只控制 HDF 设备节点加载，与这个直接链接路径完全无关。

错误的加载路径（第一次修复假设的）：
HDF devmgr → device_info.hcs → display_host → display_layer ← 实际不走这条路

实际的加载路径：
graphic_utils_lite → BUILD.gn ldflags "-ldisplay_layer" → dlopen → LayerInitialize

三、修复方案

核心思路

用一个 stub 共享库替换 SoC 原始 display_layer HAL。stub 提供与原库相同的导出接口 (LayerInitialize/LayerUninitialize)，但所有函数都是空操作 (返回成功)，不调用任何 display 硬件。

3.1 新增文件：display_layer_stub.c

路径： device/soc/hisilicon/common/hal/display/display_layer_stub.c

Stub 实现要点：

- LayerInitialize：分配 LayerFuncs 结构体，注册 11 个 no-op 函数指针
- GetLayerBuffer：分配 1920x1080x4 字节虚拟缓冲区 (防止渲染代码写空指针崩溃)
- Flush：直接返回成功 (不写 framebuffer)
- 其余函数：返回 DISPLAY_SUCCESS

```
// 关键函数示例
static int32_t StubGetLayerBuffer(uint32_t devId, uint32_t layerId, LayerBuffer *buffer)
```

```

{
    if (!buffer) return DISPLAY_NULL_PTR;
    if (!g_stubBuf) {
        g_stubBuf = calloc(1, (size_t)STUB_WIDTH * STUB_HEIGHT * STUB_BPP);
        if (!g_stubBuf) return DISPLAY_FAILURE;
    }
    buffer->width = STUB_WIDTH;
    buffer->height = STUB_HEIGHT;
    buffer->pitch = STUB_WIDTH * STUB_BPP;
    buffer->pixFormat = PIXEL_FMT_RGBA_8888;
    buffer->data.phyAddr = 0;
    buffer->data.virAddr = g_stubBuf; // 渲染代码写入此缓冲区, Flush 不做任何事
    return DISPLAY_SUCCESS;
}

```

为什么需要虚拟缓冲区： hi_fbdev.cpp 的 HiFbdevInit() 会将 GetLayerBuffer 返回的 virAddr 存为渲染表面。如果为 NULL，UI 渲染代码在写像素时会发生段错误。stub 分配一块内存让渲染代码“画到空气中”，Flush 不做任何事所以不会有任何输出。

3.2 修改 BUILD.gn: 条件编译 stub

路径： device/soc/hisilicon/common/hal/display/BUILD.gn

将 shared_library("display_layer") 的 sources 列表改为条件编译：

```

shared_library("display_layer") {
    output_name = "display_layer"
    if (board_name == "hispark_hi3519dv500") {
        sources = [ "display_layer_stub.c" ]           # ← hi3519dv500 使用 stub
    } else {
        sources = [                                     # ← 其他板使用原始 SoC 代码
            "//drivers/peripheral/display/hal/disp_hal.c",
            "display_layer.c",
            "display_overlay_layer.c",
            "vpss_resmng.c",
            "hdmi.c",
            "vo_parameter_calc.c",
            "bt1120.c"
        ]
    }
}
// 其余 include_dirs、deps、defines、ldflags 保持不变

```

为什么只改 sources： stub 不依赖 MPI 头文件，但现有的 include_dirs、defines、ldflags 不影响编译（未使用的头文件和宏定义不会报错）。MPI 库仍会链接为 DT_NEEDED 依赖，但 stub 不会调用它们，设备上这些库已部署所以运行时不报错。

3.3 修改 init.cfg：移除 display 内核模块

路径： `device/board/hisilicon/hispark_hi3519dv500/linux/system/cfg/init.Hi3516DV300.cfg`

从 `fs` job 的 `cmds` 中移除以下 4 行：

```
"insmod /vendor/modules/hi3516cv500_tde.ko",      # TDE 硬件图形加速
"insmod /vendor/modules/hi3516cv500_vo_dev.ko",    # VO 视频输出设备
"insmod /vendor/modules/hifb.ko video=\"hifb:vram0_size:16200\"", # 帧缓冲
"insmod /vendor/modules/hi3516cv500_hdmi.ko",      # HDMI 输出
```

从 `boot` job 的 `cmds` 中移除 1 行：

```
"chown system graphics /dev/graphics/fb0",
```

为什么要移除： display 已禁用，加载这些模块是浪费内存。TDE 占用内核内存，VO 设备驱动占用 MMZ 空间，HDMI 驱动占用中断资源。移除后可节省约 16MB framebuffer 内存 (`vram0_size:16200KB`) 。

3.4 修改 device_info.hcs：禁用 HDF display 设备节点

路径： `vendor/hisilicon/hispark_hi3519dv500_linux/hdf_config/device_info/device_info.hcs`

在 `display :: host` 块中，为所有 5 个设备的 `deviceNode` 添加或修改 `preload = 2`：

```
device_hdf_disp      (HDF_DISP)      → preload = 2
device_hi35xx_disp   (HI351XX_DISP)   → preload = 2
device_lcd           (LCD_ICN9700)    → preload = 2   (原为 0)
device_pwm_bl        (PWM_BL)         → preload = 2   (原为 0)
device_backlight     (HDF_BL)         → preload = 2   (原为 0)
```

为什么还要改 HCS (明明主修复在 stub)： 虽然 `display_layer` HAL 不通过 HDF 加载，但 HCS 中注册的 display 设备节点仍然会被 HDF 框架处理。`preload = 2` (`DEVICE_PRELOAD_DISABLE`) 可以阻止 HDF 框架尝试加载这些设备驱动，避免不必要的错误日志和资源消耗。这是**防御性禁用**，确保 display 在 HDF 层和 HAL 层都完全关闭。

四、修改文件清单与备份

文件	操作	备份文件
<code>device/soc/hisilicon/common/hal/display/display_layer_stub.c</code>	新增	—
<code>device/soc/hisilicon/common/hal/display/BUILD.gn</code>	修改 sources 条件编译	<code>BUILD.gn.bak</code>
<code>device/board/.../init.Hi3516DV300.cfg</code>	移除 5 行	<code>init.Hi3516DV300.cfg.bak</code>
<code>vendor/.../device_info.hcs</code>	5 个设备 preload=2	<code>device_info.hcs.bak</code>

五、恢复方法

如果需要重新启用 display 功能（例如接 HDMI 显示器调试）：

```
cd <源码根目录>

# 1. 还原 BUILD.gn (恢复 soc display_layer 编译)
```

```
cp device/soc/hisilicon/common/hal/display/BUILD.gn.bak \
device/soc/hisilicon/common/hal/display/BUILD.gn

# 2. 删除 stub 文件
rm device/soc/hisilicon/common/hal/display/display_layer_stub.c

# 3. 还原 init 配置（恢复 display 内核模块加载）
cp device/board/hisilicon/hispark_hi3519dv500/linux/system/cfg/init.Hi3516DV300.cfg.bak \
device/board/hisilicon/hispark_hi3519dv500/linux/system/cfg/init.Hi3516DV300.cfg

# 4. 还原 HCS 配置（恢复 HDF display 设备节点）
cp vendor/hisilicon/hispark_hi3519dv500_linux/hdf_config/device_info/device_info.hcs.bak \
vendor/hisilicon/hispark_hi3519dv500_linux/hdf_config/device_info/device_info.hcs

# 5. 重新全量编译烧录
rm -rf out && hb build -f
```

六、验证方法

修改完成后全量编译烧录：

```
# 全量编译验证
rm -rf out && hb build -f
```

烧录后在串口验证：

```
# 检查是否还有 HDF_DISP 刷屏
dmesg | grep "layer is not opened"    # 应无输出

# 检查内存是否正常分配
cat /proc/meminfo | head -5
# MemTotal 应约 227MB（256M - 内核保留）

# 确认 display 内核模块未加载
lsmod | grep -E "tde|vo_dev|hi_fb|hdmi"    # 应无输出
```